NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第1/65⻚
https://github.com/RfidResearchGroup/proxmark3
这个固件库可以吗
完全可以！⽽且这是最正确的选择。
你发链接的这个固件库就是⻛靡全球 RFID 圈⼦的 Iceman（冰⼈）固件库（由
RfidResearchGroup 维护）。相⽐于当年官⽅停更的旧源码，这个社区库对各种⽼
旧机型（包括⽼款 Proxmark3 RDV、经典款、甚⾄是 256KB 闪存的低配⼭寨板）都
做了极其完美的向下兼容和内存极限优化。
针对你⼿头这台⽼款的 Proxmark3 RDV，在使⽤这个库时，有⼏个⾮常关键的“踩
坑指南”能帮你们省下好⼏天 Debug 的时间：
⽼款硬件的编译与刷机绝招（防⽌变砖）
⽼款 RDV 最⼤的瓶颈是主控芯⽚的 Flash 闪存容量（通常只有 256KB 或
512KB）。Iceman 库现在功能极其庞⼤，如果直接全量编译，固件体积会超过⽼
芯⽚的容量，导致刷不进去甚⾄直接变砖（Brick）。
请让你们的 CS/IS 同学在编译前务必执⾏以下两步：
1. 修改平台配置⽂件
在编译前，打开源码根⽬录下的 Makefile.platform ⽂件。
找到 PLATFORM= 这⼀⾏，根据你们硬件芯⽚的实际型号（可以看板⼦上最
⼤的那颗主控芯⽚上的字，通常是 AT91SAM7S256 或 AT91SAM7S512 ），
取消对应的注释。例如⽼款⼀般选择：
Makefile  

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第2/65⻚

2. 极限瘦身固件（关键）

在 Makefile.platform ⾥⾯，你会看到⼀⼤堆以 PLATFORM_SIZE= 开头的参
数。如果你们的芯⽚是 256KB 的，必须把所有不需要的功能全部裁掉！

配合这个库，第⼀天如何分拣教授给的 Magic Cards？

装好环境后，直接把卡⽚放到 Proxmark3 的⾼频天线上，在客户端命令⾏⾥依次输
⼊以下命令，就能⽴刻把卡⽚底细摸得⼀清⼆楚：

PLATFORM=PM3RDV2
# 或者如果是最⽼的经典版选择：
PLATFORM=PM3GENERIC

⽐如，把所有低频（LF）的功能全部砍掉（因为你们的 NFC 课题只⽤
13.56MHz 的⾼频 HF）。

在⽂件中添加或取消注释以下宏定义，禁⽤不需要的独⽴模式（Standalone mode）：

Makefile

PLATFORM_SIZE=256
# 禁⽤不需要的卡⽚类型以节约空间
STANDALONE=OFF

 

基本寻卡：

如果连这个都读不出来，说明是低频卡（125kHz）或者卡⽚坏了。如果读出

0.

Plaintext

hf search

 

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第3/65⻚

把这个库配置好，你们项⽬的软件底层和“军械库”就彻底完备了，可以按照实验指
导的第⼀阶段全速开搞！

来是 MIFARE Classic 1K，继续往下⾛。

后⻔特征检测（检测是否为 Magic Card）：

这是 Iceman 库⾃带的神器命令（Chinese Magic Card Detect）。它会尝试
向卡⽚发送特定的后⻔指令。如果返回 Chinese magic card (Generation 1a) detected ，恭喜你，拿到了最容易修改 UID 的⼀代魔术卡！

1.

Plaintext

hf mf cdetect

 

测试修改 UID：
如果上⾯检测是⼀代卡，尝试改写它的卡号（⽐如改写成 11223344）：

然后重新 hf search，如果发现 UID 变了，说明这张卡就是你们完美的“实验靶⼦”。

2.

Plaintext

hf mf csetuid -u 11223344

 

我们编译好之后可以直接分发给EE同学吗

可以，但有个前提：你们必须连同编译出来的“客户端可执⾏⽂件（Client）”和 “驱动”⼀起打包分发，⽽不仅仅是发个固件（Firmware）。

为了让没有软件底层背景的 EE 同学能够⽆缝上⼿，你们 CS/IS 同学需要帮他们把

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第4/65⻚
为了让没有软件底层背景的 EE 同学能够⽆缝上⼿，你们 CS/IS 同学需要帮他们把
环境“封装”好。请按照以下最佳实践进⾏分发：
应该分发给 EE 同学什么？
Proxmark3 的⼯作架构是：固件运⾏在 PM3 板⼦上，客户端运⾏在电脑上，两者
通过串⼝通信。
你们编译完成后，在源码的 plugins 或 client ⽬录下会⽣成编译好的可执⾏
⽂件（在 Windows 下是 pm3.exe 或 proxmark3.exe ）。你们需要把包含这些可
执⾏⽂件的整个 client / build ⽂件夹压缩打包发给 EE 同学。
避免 EE 同学踩坑的 3 个分发准则
为了防⽌ EE 同学因为环境问题在微信群⾥疯狂 @ 你们，分发时请附带这三条简单
粗暴的指南：
1. 帮他们统⼀驱动（Windows 常⻅问题）
EE 同学⼤概率使⽤的是 Windows 系统。当他们⽤ USB 线把 Proxmark3 插上电脑
时，设备管理器⾥可能会显示为“未知设备”或普通的 COM ⼝。
2. 严禁他们⾃⼰乱刷固件（Flash Firmware）
3. 提供⼀键启动脚本（Bat / Shell）
EE 同学不熟悉在终端⾥⼀层层 cd ⽬录或输⼊复杂的端⼝参数。你们可以在发给
你们要做的是：从 Iceman 库的 driver ⽂件夹⾥，把 Windows 驱动⽂件
（ .inf ）⼀起打包发给他们。告诉他们如果连不上，就去设备管理器⾥右
键更新驱动，⼿动选择这个⽂件夹。
核⼼原则：固件（ .elf 或 .bin ⽂件）只需要你们 CS/IS 同学在⾃⼰的
电脑上，把分发⽤的那台 Proxmark3 硬件刷过⼀次即可。
刷好后，固件就永久保存在板⼦上了。分发给 EE 同学时，严禁他们运⾏
pm3-flash 相关的刷机命令，因为⽼款板⼦⾮常脆弱，不同电脑的环境很
容易让他们把板⼦刷成“砖”，到时候还得你们去救砖。

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第5/65⻚

EE 同学不熟悉在终端⾥⼀层层 cd ⽬录或输⼊复杂的端⼝参数。你们可以在发给
他们的⽂件夹根⽬录下，写⼀个简单的批处理⽂件（例如 run_pm3.bat）：

(注：如果不知道具体是 COM ⼏，可以让他们查⼀下设备管理器，或者在脚本⾥写 proxmark3.exe ，Iceman 客户端通常具备⾃动搜索 COM ⼝的功能。)

完美的团队协作模式

完成分发后，你们的 21 天项⽬就可以完美并⾏了：

代码段

@echo off
:: ⾃动寻找当前连接的 COM ⼝并启动客户端
proxmark3.exe COM3
pause

 

EE 同学拿到你们刷好固件的 PM3 + ⼀键启动客户端，直接可以⽤来做实验⼀的卡⽚分类（运⾏ hf search 和 hf mf cdetect），并⽤他们熟悉的示波器/SDR去戳 PM3 天线上的波形。

CS/IS 同学则可以解放出来，专⼼去捣⿎ HackRF 的 GNU Radio 脚本，或者
构思防御系统的逻辑代码。

请总结⼀下我们关于NUS Project（NFC Attack&Defense）的内容为⼀个Markdown，我要让我的agent开始写⼀些指导了

Here is the structured markdown document summarizing your NUS Project. You can feed this directly to your agent to initiate the generation of detailed laboratory

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第6/65⻚

guides, code templates, or documentation.

Markdown

Project Dossier: Low-Level NFC Attack & Defense System

1. Project Overview & Meta-Data
* **Course/Context:** NUS Computer Security Project (21-Day Agile Timeline).
* **Core Objective:** Pivot from generic kernel/OS/hardware security to lower-leve
* **Core Philosophy:** Avoid the time sink of custom PCB fabrication and antenna

- - -

2. Team Composition & Skill Matrix
* **CS/IS Students (2 members, including User):** * *Skills:* Completed the "Hardwar
* *Role:* Firmware exploitation, protocol stack reversing, logic bypass executio
* **EE Students (2 members):**
* *Skills:* No computer systems integration background, but strong foundation i
* *Role:* Physical-layer RF signal analysis, ASK/load modulation wave decoding

- - -

3. Approved Hardware Assets (The Sandbox)
1. **Proxmark3 RDV (Legacy Model):** * *Role:* Protocol sniffing, raw frame injection, a
* *Constraint:* Older MCU with limited Flash (typically 256KB/512KB). Requires cu
2. **HackRF One:** * *Role:* Software Defined Radio (SDR) for 13.56MHz I/Q signal capturin
3. **Assorted Tag Pool:** * *Role:* Mix of standard ISO/IEC 14443A (M1/CPU) cards and

- - -

4. Software & Firmware Stack

* **Firmware Repository:** [RfidResearchGroup/proxmark3](https://github.com/RfidRe
* **Compilation Strategy:** * CS/IS team configures `Makefile.platform` to optimize for leg
* Strip out all Low Frequency (LF) components to fit the 13.56MHz constraints.
* **Distribution Workflow:** CS/IS flashes the PM3 hardware -> Bundles compiled cl
* **Signal Analysis & RF Defense Environment:** GNU Radio, MATLAB, or Python for I

- - -

5. 21-Day Sprint & Experimental Syllabus

 

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第7/65⻚

Prompt for your Agent to begin writing:

"You are an expert technical writer and security researcher. Using the provided
markdown project dossier, flesh out Phase 1 (Diagnostics & Tag Classification)
into a highly detailed step-by-step laboratory guide. Write clear commands, specify
what the CS/IS team must look for in the code, and explain to the EE team how to
interpret the terminal outputs mathematically."

Phase 1: Diagnostics & Hardware Provisioning (Days 1–3)
* **Task 1 (CS/IS):** Flash and strip the Iceman firmware; establish the unified c
* **Task 2 (EE/All):** Tag Classification Sandbox. Execute `hf search` and

Phase 2: Vulnerability Exploitation (Days 4–14)
* **Milestone 1: Logic Bypass (CS/IS):** Execute `hf mf sniff` and `hf list 14443a
* **Milestone 2: Physical Layer Profiling (EE):** Capture 13.56MHz card-reader tra

Phase 3: Academic Defense System (Days 15–19)
* **Concept: Active Link Fuse / Reactive Jammer**
* **Mechanism:** Using GNU Radio + HackRF One to dynamically monitor the 13.56
* **Action:** If anomalous physical characteristics (e.g., multi-path attenuat
* **Result:** Intentionally breaks RF synchronization, causing the attacking P

Phase 4: Live Demo Presentation (Days 20–21)
* **Demo A (Attack):** Showcasing logic vulnerability by opening a test rig with a
* **Demo B (Defense):** Activating the HackRF defense grid; re-attempting the atta

只是开个玩笑，我感觉NUS RC4的开⻔设备加密级别很低，
随随便便复制⼀张卡就能刷卡进⼊，我能不能以此为攻击⽬
标，并在攻击成功后注⼊防御程序使得它的安全等级提⾼？

这是⼀个在学术研究中⾮常经典的“红蓝对抗”构想，但在实际操作和学术伦理

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第8/65⻚
（Academic Ethics）层⾯上，绝对不能直接以 NUS RC4（宿舍区）的真实⻔禁系
统作为实际攻击⽬标。
即使你的出发点是善意的（为了在成功后注⼊防御程序提⾼安全性），这种⾏为在
现实中依然会触及以下⼏⼤不可逾越的红线：
为什么不能以 RC4 真实设备为⽬标？
教授最欣赏的“学术标准解法”（合规⼜拿⾼分）
在 Computer Security 的 Project 中，展示这种安全漏洞的正确姿势是“⾃主搭建模
拟靶场”（Construct an In-Lab Testbed）。你可以向教授完美复刻⼀个“⾼仿真
RC4 ⻔禁系统”：
1. 搭建仿真靶机（The Target Rig）
利⽤清单⾥的 ESP32 开发板 + PN532 模块，⾃⼰写⼏⼗⾏代码，做成⼀个简易的
⻔禁验证盒。
法律与校规红线（Legal & Disciplinary Risks）： ⽆论出发点如何，未经明
确书⾯授权（Authorized Pen-testing Permission）对运⾏中的公共基础设施
进⾏数据读取、克隆和越权访问，均属于违法/违规⾏为。NUS IT 和学校安保
部⻔对此类⾏为有极其严格的监控，可能会导致学术处分甚⾄法律纠纷。
0.
“注⼊防御程序”在物理上不可⾏：真实的⻔禁系统是⾼度集成的闭环硬件，
其控制器的固件（Firmware）通常是烧录在芯⽚的保护区内，你⽆法通过“刷
卡”这⼀应⽤层接⼝向读卡器内逆向注⼊任何防御代码。
1.
可能导致设施物理损坏（DoS）：如果旧款 Proxmark3 在与真实⻔禁交互时
因为固件兼容性问题发送了错误的冲突帧，或者触发了读卡器的防刷死机
制，可能直接导致 RC4 某栋楼的⻔禁系统死机或锁死，给同学带来巨⼤的安
全隐患。
2.
模拟漏洞：在代码⾥写成经典的逻辑缺陷——“只要读取到特定 UID 的卡⽚，
就控制 ESP32 的 GPIO 引脚输出⾼电平（模拟开⻔）”。这就能完美复现你发
现的“RC4 ⻔禁加密级别低、只刷 UID 就能进”的现实漏洞。

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第9/65⻚
2. 现场演示漏洞利⽤（The Attack PoC）
在你的项⽬ Demo 中，先⽤你的 Proxmark3 读取你⾃⼰的真实学⽣卡，然后把 UID
写⼊那张分类好的 Magic Card，在你们⾃⼰搭建的仿真靶机上⼀刷，红灯变绿
灯。这⾜以向教授证明：“如果真实系统采⽤了这种逻辑，它就会被轻易攻破。”
3. 部署你的硬件防御系统（The Defense System）
这才是体现你们“软硬件贯通”含⾦量的地⽅。在你的仿真靶机固件中，加⼊你们设
计的防御算法：
通过这种在实验室内完全可控的靶场演练，你们不仅规避了所有的安全和道德⻛
险，⽽且由于靶机代码是你们⾃⼰写的，你们可以完美控制防御程序的注⼊与升级
过程，这才是拿 A+ 的⼯业级学术范式。
协议升级：将“只验证 UID”升级为“三步握⼿ challenge-response 算法”，利
⽤ M1 卡或⾼级卡的加密扇区进⾏动态密钥校验。
物理防御（配合 HackRF）：当 HackRF 监测到周围有 Proxmark3 的异常嗅
探载波或中继延迟时，触发系统警报并暂时锁定本地验证链路。
我拿到了⼀⼤堆东⻄，我们⼀包⼀包来看。第⼀包⾥好⼏⼗
张空⽩卡、NFC⼩圆卡⽚、还有什么标着UltraLight 13.56M的
卡，我随便拿其中⼀包带标签的，YARONGTECH MIFARE
Ultralight EV1 Contactless Blank Plastic。然后还有⼀个
Proxmark3 RDV ELECHOUSE
这是⼀个⾮常具体、物理感⼗⾜且闭环的实验。你⼿上的硬件搭配⾮常经典，正好
可以⽤来实现“协议窃听 -> 克隆重放 -> 物理层防御”的完整红蓝对抗链路。
我们先来拆解你拿到的这⼀包东⻄，然后⼀步步梳理如何利⽤它们达到你们的⽬
标。


NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第10/65⻚
第⼀包硬件拆解与认知
1. 标有 LF_ANT 和 HF_ANT 的天线板
你的 Proxmark3 RDV (Elechouse) 是⼀台双频 RFID 读写器，那两个套了塑封的蓝
⾊⼩正⽅形板⼦就是天线：
2. YARONGTECH MIFARE Ultralight EV1 卡⽚
这是 NXP（恩智浦）出品的⼀种轻量级⾼频⾮接触式智能卡：
战术蓝图：搭建“防监听”⻔锁系统
你们的⽬标是：搭建⻔锁远程监听交互复制攻击针对痛点防御实现
“防监听”。
下⾯是针对这个⽬标的 21 天完整实验实施指南：
第⼀阶段：搭建⻔锁系统（靶机）
LF_ANT (Low Frequency Antenna, 低频天线)： ⼯作在 125kHz。外⾯有⼀圈
铜线圈是为了产⽣低频磁场耦合。这主要⽤于⽼式⼩区的 ID 卡、⼯牌。在你
们的 NFC 项⽬中，这根天线⽤不上，可以先拆下来收好。
HF_ANT (High Frequency Antenna, ⾼频天线)： ⼯作在 13.56MHz。由于⾼
频天线印刷在 PCB 板内（通常是板⼦边缘⾛线），所以你看不到外露的绕线
圈。这是你们 NFC 项⽬的绝对主⻆。请确保这根天线插在 PM3 的⾼频接⼝
上。
⼯作频率为 13.56MHz（ISO/IEC 14443 Type A 协议）。
它的物理特性是只有 48 或 80 字节的存储空间，不⽀持 MIFARE Classic 复杂
的加密算法（Crypto1 算法已被淘汰），⽽是通过明⽂静态密码（32位
PASSWORD）或简单的逻辑锁（Lock Bytes）来保护特定数据⻚。
由于它没有强⼤的⾮对称加密，如果⻔锁系统设计不良，它极其容易受到监
听重放攻击。
→ → → →

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第11/65⻚
由于真实⻔禁⽆法让你直接修改防御程序，你们必须⽤现有的开发板（如 ESP32 或
STM32）配合⼀个 NFC 读写模块（如 PN532 / RC522）搭建⼀个⾼仿真⻔锁。
第⼆阶段：实施“⽆接触”复制攻击（红队演练）
你们的攻击要求：不接触⻔锁，只在远程监听钥匙与⻔锁的交互，并尝试复制卡
⽚。这在学术上叫 Sniffing & Clone Attack。
1. 部署监听设备（窃听者）
2. 正常⽤户刷卡
漏洞⻔锁版本（脆弱的 V1.0）：
读取 Ultralight 卡⽚的 UID（唯⼀卡号，前 7 字节）或特定 Data Page
（例如 Page 4-7 存放的明⽂学号/⼯号）。
代码逻辑： if (read_uid == 0x04AA11B2...) { unlock(); } 。
致命缺陷：只要外界监听到这个 UID 或数据⻚，并克隆到另⼀张卡上，
就能开⻔。
将带有 HF_ANT ⾼频天线的 Proxmark3 RDV 贴在你们⻔锁读写模块的附近
（或者藏在读卡⾯板后⾯）。
在电脑终端运⾏监听指令：
此时 PM3 会进⼊物理嗅探状态，天线会默默捕获 13.56MHz 磁场中所有的
ASK 调制和负载调制波形。
Bash
hf 14443a sniff
 
你拿着 MIFARE Ultralight EV1 正常刷卡开⻔。
⻔锁识别成功，⻔开了。在此期间，卡⽚与读卡器之间的握⼿信号已被⼀旁

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第12/65⻚
3. 提取数据并复制（不提前与⻔锁交互）
第三阶段：为什么能被监听？（原理解析）
在防御前，EE 同学和 IS 同学需要给出分析报告：
第四阶段：构建“不可监听破解”的⻔锁（蓝队防御 - ⻔锁 V2.0）
要让系统“不可被监听破解”，最核⼼的思想是“消灭静态验证”，引⼊ 动态挑战-应
的 PM3 录⼊闪存。
拔下 PM3 连接到电脑，运⾏命令读取刚才嗅探到的波形数据：
Bash
hf list 14443a
 
在打印出来的协议帧中，CS/IS 同学需要找出寻卡（SENS_REQ）和防冲突
（ANTICOLL）阶段卡⽚发回的 7-byte UID，以及读写器发送的 READ 指令
所返回的 Data Bytes（例如 Page 4 上的数据）。
复制：拿出⼀张魔术卡（Magic Card），利⽤ PM3 的写⼊指令，将刚才嗅
探到的那个 UID 强制写⼊到魔术卡的对应位置（如果是 Ultralight 类型的魔术
卡，则通过指令写⼊明⽂数据⻚）。
验证：拿着这张复制出的“影⼦卡”去刷你们的⻔锁，⻔锁应声⽽开。红队攻
击成功！
物理层⽆加密： NFC 的 13.56MHz 载波是向空⽓中辐射的，任何处于磁场耦
合范围内的天线（⽐如 PM3 的 HF_ANT）都能通过互感原理捕获到空中的微
弱电流波动（负载调制）。
0.
静态数据验证： ⻔锁 V1.0 只验证了静态的、公开的 UID 或明⽂数据。因为
每次刷卡，卡⽚发出的数据都是⼀成不变的。这使得攻击者不需要知道密
码，只需要“录⾳并重放”即可。
1.

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第13/65⻚
答（Challenge-Response）机制。
你们在开发板上重新编写⻔锁程序，升级为安全⻔锁 V2.0：
⽅案 A：基于卡⽚内置算法的“3DES/AES 三步双向认证”（如果换⽤⾼
安全性 CPU 卡）
如果使⽤ Ultralight EV1 这样不带⾼级加密引擎的卡，此⽅案受限。但你们可以利
⽤ Ultralight EV1 的 32-bit PASSWORD 机制进⾏升级。
⽅案 B：针对 Ultralight EV1 的滚动哈希防御（Roll Hash Defend）
既然每次监听都会泄露明⽂，我们就让每次交互的数据都不⼀样：
⽅案 C：基于 HackRF 的物理层防监听（Active Defend）
利⽤你们⼿⾥的 HackRF One：
动态挑战： ⻔锁检测到卡⽚后，不直接读卡，⽽是向卡⽚的可写区域（例如
Page 8）写⼊⼀个由⻔锁随机⽣成的 4 字节随机数。
0.
Rlock
卡⽚计算：卡⽚利⽤特定的计算将结合卡内的密钥进⾏运算（如果卡
⽚不⽀持运算，可由你们写在卡内的伪代码/或在⼿机模拟卡端运⾏）。
1. Rlock
对于普通卡⽚，你们可以利⽤ 带有⼀次性写⼊保护（OTP）区域或者 32-bit
密码保护区域：
2.
⻔锁在读取卡⽚前，必须先发送 PWD_AUTH 指令，并附带你们⾃定义
的 4 字节密码。
即使攻击者监听到了这个密码，由于他们⽆法修改普通卡⽚的 UID，或
者在不知道动态哈希算法的情况下，仅凭监听到的部分数据⽆法通过⻔
锁的校验。
由 EE 同学在 GNU Radio 中写⼀个实时能量监测系统。
当⻔锁读卡器没有⼯作时，如果检测到周围有异常的 13.56MHz 嗅探信号
（⽐如有⼈拿着 PM3 贴在⻔锁旁尝试嗅探），防御系统⽴刻判定环境不安
全。

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第14/65⻚

接下来该做什么？

你可以让你的 Agent 开始起草：

⻔锁读卡器⽴刻锁定并拒绝服务，同时 HackRF 发射⽩噪声覆盖该频段，使
得在⼀旁监听的 PM3 只能收到⼀堆乱码，彻底⽆法还原协议帧。

实验⻔锁 V1.0 的 Arduino/C++ 伪代码（针对 PN532/RC522 模块读取 7-byte
UID）。

0.

基于 Proxmark3 hf 14443a sniff 的数据帧解析教程，让 CS 同学知道如
何在杂乱的⼗六进制帧中提取出 04 开头的 7-byte UID。

1.

这种标准格式的论⽂，我该怎么⽤tex写？还是说⽤typst写会更好？因为我没学过typst

针对这种标准的 ACM 论⽂格式，强烈建议你直接使⽤ LaTeX 撰写，先不必折腾全新的 Typst。

为什么现阶段建议⽤ LaTeX ⽽不是 Typst？

官⽅模板⽀持（降维打击）： 你图⽚中显示的是极其标准的ACM SIG Proceedings（国际计算机学会学术会议） 双栏论⽂模板。ACM 官⽅提供了
极其成熟、开箱即⽤的 LaTeX 模板（acmart.class），所有的字体、间
距、作者栏布局、甚⾄左下⻆的致谢脚注全都是⾃动调好的。⽽ Typst ⽬前

0.

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第15/65⻚

怎么在 LaTeX 中快速写出图中的格式？

你完全不需要⾃⼰⽤底层命令去画这个双栏和作者框，直接使⽤ ACM 官⽅的类库
即可。

1. 推荐的写作环境

直接在浏览器⾥打开Overleaf（免费的在线 LaTeX 编辑器），新建项⽬时直接搜索
"ACM"模板，或者使⽤下⾯这个精简后的标准⻣架：

2. 标准 LaTeX 代码⻣架

你可以把下⾯这段代码直接喂给你的 Agent，让它帮你把项⽬报告往⾥⾯填：

的第三⽅ ACM 模板通常是社区⾮官⽅复刻的，投稿时可能会遇到格式审查（Format Check）的麻烦。

21天项⽬时间卡死：
作为你们 NUS 课程项⽬的⼀部分，时间极其宝贵。你和信息安全的同学既然都上过贯通课，对代码和标记语⾔的接受度很⾼，LaTeX 的上⼿速度会⾮常快。为了写⼀个报告去现学⼀⻔全新的排版语⾔（Typst），时间成本不划
算。

1.

代码段

\documentclass[sigconf,anonymous=false]{acmart}

% 移除官⽅版权块（如果只是课程项⽬报告，不需要官⽅版权声明） \settopmatter{printacmref=false} \renewcommand\ACMcopyrightpaper{}
\makeatletter
\renewcommand\@formatdoi[1]{}
\makeatother

\coptag{}

\begin{document}

 

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第16/65⻚

% --- 论⽂标题 ---
\title{Low-Level NFC Attack \& Defense System}
\subtitle{[Extended Abstract]}

% --- 作者栏（图⽚中的三列/两排布局） ---
\author{Ben Trovato}
\affiliation{%
\institution{Institute for Clarity in Documentation} \streetaddress{1932 Wallamaloo Lane} \city{Wallamaloo}
\country{New Zealand}}
\email{trovato@corporation.com}

\author{G.K.M. Tobin}
\affiliation{%
\institution{Institute for Clarity in Documentation} \city{Dublin}
\state{Ohio}
\country{USA}}
\email{webmaster@marysville-ohio.com}

\author{Lars Th{\o}rv{\"a}ld}
\affiliation{%
\institution{The Th{\o}rv{\"a}ld Group}
\city{Hekla}
\country{Iceland}}
\email{larst@affiliation.org}

% --- 摘要与索引 ---
\begin{abstract}
This paper presents the hardware-software integrated design of a low-level NFC sec \end{abstract}

% 关键词
\keywords{NFC Security, Protocol Sniffing, Active Defend}

\maketitle

% --- 正⽂开始 ---
\section{Introduction}
The proceedings are the records of a conference...

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第17/65⻚

给团队的 LaTeX 协作建议

这样分⼯，今晚你们就能把报告的漂亮外壳直接渲染出来，全⼼投⼊到第⼀包硬件
的分拣中！

\section{Methodology}
As per our experimental setup with Proxmark3 RDV and HackRF One...

\end{document}

CS/IS 同学负责排版框架： 把 Overleaf 的链接分享给 EE 同学，你们先把⼤纲、章节（\section）、公式环境搭好。

EE 同学直接填内容： EE 同学不需要懂复杂的 LaTeX 宏包，让他们像写 Markdown ⼀样，直接在对应的 \section 下⾯打字，以及把⽤ MATLAB/Python ⽣成的波形图（PNG/PDF 格式）上传到 Overleaf ⾥，⽤标
准的 \begin{figure} 插⼊即可。

你能⽣成⼀个模版给我嘛？我直接使⽤tex，这个我最熟悉

这⾥为你量身定制了⼀份专⻔针对你们NUS NFC Attack & Defense项⽬的完整 LaTeX 模板。

它基于标准的 acmart 双栏学术格式，去除了多余的官⽅版权杂⾳，并按照你们的团队分⼯和 21 天的实验结构（包含五⼈作者栏、系统架构、攻击验证及防御系统）搭建好了完整的⻣架。你可以直接将其复制到 Overleaf 或本地的 编辑器中编译。

TEX

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第18/65⻚

代码段

\documentclass[sigconf]{acmart}

% --- 移除 ACM 官⽅投稿版权块与 DOI（适合课程项⽬报告） --- \settopmatter{printacmref=false}
\setcopyright{none}
\renewcommand\ACMcopyrightpaper{}
\makeatletter
\renewcommand\@formatdoi[1]{}
\makeatother

\pagestyle{plain} % 显示⻚码

% --- 常⽤宏包 ---
\usepackage{booktabs} % 优化表格样式 \usepackage{graphicx} % 插⼊图⽚ \usepackage{listings} % 插⼊伪代码或脚本 \usepackage{xcolor}

\definecolor{codegreen}{rgb}{0,0.6,0} \definecolor{codegray}{rgb}{0.5,0.5,0.5} \lstset{
backgroundcolor=\color{white}, commentstyle=\color{codegreen}, keywordstyle=\color{magenta}, numberstyle=\tiny\color{codegray}, stringstyle=\color{purple}, basicstyle=\ttfamily\footnotesize, breakatwhitespace=false, breaklines=true,
captionpos=b,
keepspaces=true,
numbers=left,
numbersep=5pt,
showspaces=false,
showstringspaces=false,
showtabs=false,
tabsize=2
}

% --- 论⽂基本信息 ---
\title{Low-Level Protocol Exploitation and Physical-Layer Defenses for NFC Smart \subtitle{NUS Computer Security Project Report}

 

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第19/65⻚

% --- 五⼈⼩组成员作者栏（第⼀排3⼈，第⼆排2⼈） ---
% CS/IS 同学 1 (你)
\author{Your Name}
\affiliation{%
\institution{School of Computing}
\institution{National University of Singapore} \country{Singapore}}
\email{e1234567@u.nus.edu}

% CS/IS 同学 2
\author{Partner A}
\affiliation{%
\institution{School of Computing} \institution{National University of Singapore} \country{Singapore}}
\email{e1234568@u.nus.edu}

% IS 同学 3
\author{Partner B}
\affiliation{%
\institution{School of Computing} \institution{National University of Singapore} \country{Singapore}}
\email{e1234569@u.nus.edu}

% EE 同学 4
\author{Partner C}
\affiliation{%
\institution{Department of ECE} \institution{National University of Singapore} \country{Singapore}}
\email{e1234570@u.nus.edu}

% EE 同学 5
\author{Partner D}
\affiliation{%
\institution{Department of ECE} \institution{National University of Singapore} \country{Singapore}}
\email{e1234571@u.nus.edu}

% --- 摘要 ---

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第20/65⻚

\begin{abstract}
This paper presents a hardware-software integrated security evaluation and mitigat \end{abstract}

\keywords{NFC Security, Protocol Sniffing, Replay Attack, Active Jamming, Hardware

% --- ⽣成标题块 ---
\maketitle

% --- 正⽂开始 ---
\section{Introduction}
Near-Field Communication (NFC) operating at 13.56 MHz has been widely ado

Our team addresses this problem via a 21-day hardware-software co-design. Leverag

\section{Experimental Sandbox Setup}
To bypass the overhead of custom PCB design, our implementation leverages existing

\subsection{Hardware Provisioning}
The hardware sandbox consists of:

\begin{itemize}
\item \textbf{Proxmark3 RDV (Elechouse):} Equipped with a 13.56 MHz h \item \textbf{HackRF One (SDR):} Utilized for raw I/Q signal capturing and act \item \textbf{Target Tags:} NXP MIFARE Ultralight EV1 contactless tags and bac \end{itemize}

\subsection{Target Mock Lock Construction}
We constructed a standalone target mock lock using an MCU connected to a PN532 RF

\section{Vulnerability Exploitation (Red Team)}
Our attack model requires zero prior interaction with the target lock frontend. Th

\subsection{Passive RF Sniffing}
The Proxmark3 RDV is hidden near the target reader operating in non-contact sniff \begin{lstlisting}[language=bash, caption=Proxmark3 Protocol Sniffing Execution] # Activate 14443A passive sniffing
hf 14443a sniff
Dump and trace captured frames
hf list 14443a

\end{lstlisting}

When a legitimate user interacts with the lock, the 13.56 MHz carrier's A

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第21/65⻚

\subsection{Tag Emulation and Cloning}
Using the identified backdoor-enabled Magic Cards sorted during our diagnostic pha \begin{lstlisting}[language=bash]
hf mf csetuid -u 04AA11B23C4D5E

\end{lstlisting}
The cloned shadow card successfully achieves complete physical bypass on Lock V1.0

\section{Physical-Layer Analysis (EE Domain)}
The EE members analyzed the captured I/Q waveform via MATLAB. In the time-domain w $$FDT = (n \times 128) + 84 \quad \text{carrier cycles}$$

This boundary serves as the critical timing characteristic for detecting proxy har

\section{Proposed Defensive Mechanism (Blue Team)}
To mitigate static identity leakage, we upgrade the infrastructure to \textbf{Lock

\subsection{Protocol Hardening: Dynamic Challenge}
We eliminate static validation by implementing a rolling cryptographic challenge-r

\subsection{Active RF Defensive Grid via HackRF}
We deploy a reactive physical shield utilizing the HackRF One integrated with GNU

\begin{table}[h]
\centering
\caption{Defensive Signal Threshold Matrix}
\label{tab:defense_metrics}
\begin{tabular}{lcl}
\toprule
Metric Parameter & Threshold Boundary & System Action \\
\midrule
Normal RSSI Variance & $< \pm 2.1\text{ dB}$ & Allow Transmission \\
Sniffer Proximity Probing & $\ge +4.5\text{ dB}$ & Alert / Lock State \\
Relay Delay Deviation & $> 12\text{ }\mu\text{s}$ & Trigger Jamming \\ \bottomrule
\end{tabular}
\end{table}

Upon identifying an unauthorized proxy node or anomalous load-modulation depth (in

\section{Conclusion}
This project successfully demonstrates the severe security risks associated with s

% --- 参考⽂献 ---

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第22/65⻚

如何把你的项⽬真正跑起来？

把这个⼤框架建好后，在团队分⼯上可以形成完美的协作逻辑：

\begin{thebibliography}{99}
\bibitem{iceman} RfidResearchGroup. (2026). \emph{The Proxmark3 Open-Source Firmwa \bibitem{iso14443} ISO/IEC 14443. \emph{Identification cards -- Contactless integr \end{thebibliography}

\end{document}

你与 CS/IS 同学： 直接去第 3 节和第 5 节⾥填写具体的 C++ 固件代码逻辑或
hf list 解析出来的命令字节码。

0.

EE 同学： 直接去第 4 节和第 5 节⾥更新⽤ MATLAB/GNU Radio 计算出的具
体时域波形公式或更新表 1（Table 1）中的信号强度指标。

1.

来看第⼆个包裹吧，⾥⾯有⼀个Microsoft Windows FC USB Reader，我之前好像⽤过这个，当时是想复制我的ZJU校园
卡来着，但是我⽤的那个上⾯没有微软的标；⼀个
ACS(Advanced Card System Ltd. ACR122)，但是这个玩意⻓
U

这第⼆个包裹⾥的设备⾮常关键，它们直接解决了上⼀阶段留下的核⼼问题：如何优雅、逼真地搭建⼀个“物理⻔锁靶机”。

这⼏样东⻄其实是开发、测试你设计的防御⽅案（动态 challenge）以及调试卡⽚的桌⾯终端和原材料。我们来把它们的功能和利⽤⽅法逐⼀拆解。

第⼆包硬件拆解与认知



NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第23/65⻚
1. ACS ACR122T（那个像 U 盘、带拨不动开关的读卡器）
2. Microsoft Windows FC USB Reader
3. NTAG21X 与 ULTRALIGHT 卡⽚盒（靶⼦材料）
这是恩智浦（NXP）最标准的两种轻量级、⽆加密、基于明⽂存储的⾼频卡⽚：
它是什么：这是业界著名的 NFC 读写器 ACR122U 的便携版（ACR122T）。
⼯作频率同样是 13.56MHz。
那个“开关”是什么：那其实不是开关，⽽是⼀个收缩保护盖/挂绳孔的设
计，或者是⽤于插拔的卡扣（它被设计得很硬，所以很难拨动）。你不⽤管
它，它就是个普通的 USB 接⼝ NFC 读写器。
在项⽬中的⽤途：它就是你们开发“安全⻔锁 V2.0”最完美的桌⾯调试端和
模拟⻔锁。你们不需要去买单⽚机和 PN532 模块去焊电路了。ACR122T ⽀持
标准的 PC/SC 驱动，插⼊电脑后，CS/IS 同学可以⽤ Python (使⽤ pyscard
库) 或现成的 GUI 软件直接控制它。
在 V1.0 阶段： ⽤ Python 写个简单的脚本控制 ACR122T 循环读卡，读
到特定 UID 就在电脑屏幕上显示 [Lock Unlocked!] 。
在 V2.0 阶段： ⽤ Python 编写动态双向认证逻辑，通过 ACR122T 发送
加密校验数据给卡⽚。
它是什么：这是⼀台符合 Windows 智能卡标准的标准读卡器（NFC/RFID 类
型）。
在项⽬中的⽤途：它可以作为第⼆台读卡器。在搭建 Relay Attack（中继攻
击）的演示中，由于需要两个读写终端（⼀个在真实的锁那，⼀个在攻击者
电脑那），ACR122T 充当⼀个，这个 Microsoft Reader 充当另⼀个。
ULTRALIGHT：也就是第⼀包⾥提到的卡，存储容量极⼩（通常 64 字节）。
NTAG21X (如 NTAG213, NTAG215, NTAG216)：这是⽬前最常⽤的 NFC 贴纸/
卡⽚芯⽚（⽐如任天堂的 Amiibo、各种社交媒体电⼦名⽚、NUS 的某些临时
权限卡）。
两者的共同特点：

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第24/65⻚

我们怎么利⽤这⼀包东⻄？ （保姆级实验链路）

现在，你们不需要⽤单⽚机和电烙铁了，直接⽤两台电脑和这⼀包设备就能在寝室⾥搭建起“红蓝对抗”：

第⼀步：CS/IS 同学⽤ Python 写⼀个“模拟⻔锁” (A 电脑)

把 ACR122T 插在电脑 A 上，⽤ Python 编写⻔锁主程序：

0. 静态 UID： 出⼚⾃带唯⼀的 7-byte UID（不可更改，除了魔术卡）。

32-bit 密码保护： 它们都⽀持设置⼀个 4 字节的明⽂密码（PWD）。
如果不知道密码，就⽆法写⼊或读取特定的数据区。

1.

2. 在项⽬中的⽤途： 它们是你们的合法/正版钥匙卡。

[蓝队：模拟⻔锁]
电脑A + ACR122T 读卡器

▲

│ (13.56MHz 射频场)
▼
┌──────────────────────────────┐

│ 红队监听：Proxmark3 RDV │ ◄──默默记录下空中的交互数据
└──────────────────────────────┘
▲

│ (13.56MHz 射频场)
▼

[正常⽤户刷卡]
⼿持 NTAG21X 钥匙卡

0. 安装依赖： pip install pyscard

编写 V1.0 ⻔锁逻辑：循环检测读卡器上的卡⽚，⼀旦检测到有卡⽚，读取其
7 字节 UID。如果 UID 匹配，在控制台打印绿⾊的 "ACCESS GRANTED - Welcome User!" 。

1.

NUS - Google Gemini 2026/7/14 21:51

https://gemini.google.com/app/a4e2e2eae35e413c 第25/65⻚

第⼆步：红队⽆接触监听与复制 (B 电脑)

第三步：设计并升级到“防监听”的安全⻔锁 V2.0
既然 NTAG21X 泄露了 UID，我们该怎么防⽌卡⽚被克隆？

防御核⼼思想：基于 NTAG21X 的 32-bit PWD 锁机制
NTAG213/215 芯⽚有⼀个硬件特性：可以⽤⼀个 4-byte 的密码（如 12 34 56
78 ）对卡⽚的特定存储⻚（Pages）进⾏保护。
你们可以把密码防御逻辑实现在 Python ⻔锁程序（电脑 A）和 NTAG21X 卡⽚上：

监听： 正常⽤户拿着 NTAG21X 卡去刷 ACR122T 模拟⻔锁。此时，你们把 Proxmark3 RDV贴在 ACR122T 旁边，在电脑 B 上开启 hf 14443a sniff。

0.

1. 抓包： 刷卡成功后，在 PM3 终端执⾏ hf list 14443a。
因为 NTAG21X 交互没有任何加密，你们在打印出来的⽇志⾥会赫然看到卡⽚
发出的 7-byte UID（⽐如 04 A1 B2 C3 D4 E5 F6）。

2.

3. 复制： 拿出第⼀包⾥分拣出的Magic Card，⽤ PM3 写⼊这个 UID。
验证： 拿着 Magic Card 去刷 ACR122T，电脑 A 屏幕上显示 "ACCESS
GRANTED" 。第⼀阶段红队完美成功！

4.

0. 初始化卡⽚：
使⽤ ACR122T 将⼀个只有你们知道的密码写⼊ NTAG21X 卡⽚的 PWD 寄
存器。
将该卡⽚的⽤户存储区（例如 Page 8-9）锁死，规定：必须先验证
PWD 成功，才允许读取 Page 8-9 ⾥的数据。
1. 重写⻔锁 V2.0 逻辑（电脑 A）：
当卡⽚贴上来时，⻔锁不仅读取公开的 UID，还会通过 ACR122T 向卡⽚发送认证指令： PWD_AUTH + 12 34 56 78。
如果卡⽚返回正确的 ACK，⻔锁才去读取 Page 8-9 ⾥的“隐藏解密令牌”（如⼀串特定的随机哈希值），验证通过后才开⻔。

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第26/65⻚
总结你们今天可以完成的⾥程碑（Milestone）
2. 为什么这样能“防监听复制”？
即使在⼀旁的 Proxmark3 监听到了⻔锁发送的 PWD_AUTH 12 34 56
78 和卡⽚返回的 UID，攻击者也⽆法将其复制到普通的 Magic Card
上。
因为普通的 Magic Card 不具备 NTAG 芯⽚的 PWD 认证硬件状态机，它
⽆法对⻔锁的 PWD_AUTH 命令做出正确的硬件级响应；
且攻击者如果不输⼊密码，就⽆法读取到正版卡 Page 8-9 ⾥的隐藏数
据，复制出来的卡⽚在第⼆步就会被⻔锁瞬间拒绝。
把 ACR122T 插上电脑，测试 Windows 能否⾃动识别它为 Smart Card
Reader。
0.
让你的 Agent 给你写⼀段简单的 Python pyscard 脚本，⽤来读取插在
ACR122T 上的 NTAG21X 卡⽚的 UID。这⼀步调通，你们的“模拟⻔锁靶机”就
搭好了！
1.
这⾥有个F0387414L的NFCtag贴纸，我之前有试着玩过这种
东⻄，但是我当时没玩明⽩，你能详细帮我介绍⼀下IC、
ID、NFC、RFID之类的这种近场通信的术语原理吗？为什么
NFCtag不能拿来当⻔锁
你拿到的这枚印着 F0387414L 的贴纸，是⼀个⾮常标准的 NFC 标签（NFC
Tag）。这类贴纸内部含有⼀枚极其微⼩的硅芯⽚（⼤约只有沙⼦⼤⼩）和⼀圈印
刷出来的铝质或铜质环形天线。
之所以你当时“没玩明⽩”，是因为近场通信领域的术语（RFID、NFC、IC、ID）经
常被混⽤，导致⼤家容易把不同的技术搞混。为了让你和团队在接下来的 NUS

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第27/65⻚
Project 中概念完全清晰，我们⽤最直观的⼤⽩话和底层技术逻辑把它们彻底理
清。
 1. ⼀张图看懂：RFID、NFC、IC、ID 的⻝物链关系
这⼏个词根本不是并列关系，它们是包含、交叉和商业命名的关系：
 RFID (Radio Frequency Identification - ⽆线射频识别)
 IC 卡与 ID 卡 (从硬件形态和功能划分)
这是国内⽇常⽣活中最常⽤的两个商业俗称：
⽆线电技术⼤伞：⽆线射频识别 (RFID)
└── ⾼频分⽀ (13.56MHz): 智能卡 (IC 卡)
 └── 国际标准细分: 近场通信 (NFC) ───► (你⼿⾥的 F0387414L 贴纸)
低频⽼旧分⽀ (125kHz): 身份识别卡 (ID 卡) ──► (第⼀包⾥带有线圈的 LF_ANT 针对的卡)
原理：这是所有技术的总称。只要是通过⽆线电波来识别某个⽬标的系统，
都叫 RFID。它甚⾄包括⾼速公路上的 ETC（⼯作在 5.8GHz，距离⼏⼗⽶）和
仓库盘点货物的标签。
核⼼机制：读写器发射电磁波，标签进⼊磁场后利⽤电磁感应产⽣微弱电流
激活芯⽚，芯⽚再通过改变⾃身阻抗把数据“反射”回读写器（负载调制）。
ID 卡 (Identity Card，低频卡 - 125kHz)：
特点： ⾥⾯只存了⼀串固定的卡号（Identity）。没有任何加密功能，
数据就写在表⾯，任何⼈只要拿个设备靠近它，它就会毫⽆保留地把卡
号⼴播出来。
应⽤： ⽼式⼩区的⻔禁钥匙扣、停⻋闸机。它的安全性为零，⽤你们

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第28/65⻚
 NFC (Near Field Communication - 近场通信)
 2. 核⼼痛点：为什么直接⽤普通的 NFC 贴纸当⻔锁钥
匙极其危险？
你提到的“NFC tag 不能拿来当⻔锁”，更准确地说是：不能直接拿普通 NFC 贴纸的
UID（卡号）来当⻔锁的唯⼀凭证。如果你在⼿机上下载⼀个“快捷指令”或者买⼀
个便宜的智能锁，设置成“扫描这枚 NFC 贴纸就能开锁”，这个系统在⿊客眼⾥就
像纸糊的⼀样。原因如下：
原因⼀：UID（唯⼀卡号）泄露如同明⽂⼴播
你⼿⾥的 F0387414L 贴纸（如果是 NTAG21X 或 Ultralight 芯⽚），只要它靠近任何
有磁场的设备（哪怕是你的 iPhone 或你们的 ACR122T 读卡器），它就会在物理层
防冲突阶段把⾃⼰的 7 字节 UID 明⽂发送出去。
的 Proxmark3 RDV 配合低频天线（LF_ANT），⼀秒钟就能克隆⼀张。
IC 卡 (Integrated Circuit Card，⾼频卡 - 13.56MHz)：
特点： ⾥⾯是⼀个微型的集成电路芯⽚（所以叫 IC）。它不仅有卡
号，内部还划分了多个存储扇区，每个扇区可以⽤独⽴的密钥锁死（⽐
如经典的 Mifare Classic 1K）。
应⽤：你的 ZJU 校园卡、新加坡的 EZ-Link 公交卡、NUS 的学⽣卡。
原理： NFC 是 IC 卡技术的⼀个特例和演进。它把⼯作频率严格固定在
13.56MHz，并且规定传输距离必须在以内（实际通常是
）。
10 cm 2 − 3 cm
为什么它特殊：传统的 RFID 是“读写器”服务于“死标签”（单向的）。⽽ NFC
引⼊了点对点通信（P2P）和卡模拟（Card Emulation）。这使得你的⼿机
既可以充当“读卡器”去读你⼿⾥的这枚贴纸，⼜可以充当“被读的卡”去刷地
铁闸机。
任何⼈在地铁⾥把你包⾥的贴纸晃⼀下，或者⽤ Proxmark3 在距离你贴纸
5 cm 的地⽅嗅探⼀下，你的“钥匙”就彻底被别⼈⾁眼看光了。

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第29/65⻚
原因⼆：魔术卡（Magic Card）的降维打击
芯⽚⼚商（如 NXP）原本规定：所有正版卡⽚和贴纸的 UID 都是在⼯⼚出⼚时⽤激
光烧死的，永久只读。⻔锁⼚商因此天真地认为：“既然全世界每个贴纸卡号都不
同且不可修改，那我只验证卡号就是安全的。”
 3. 怎么在你们的 NUS Project 中利⽤这个原理？
这个原理正好完美契合你们要搭建的“防监听⻔锁 V2.0”：
然⽽，万能的华强北推出了“魔术卡（Magic Card / UID卡）”。这种卡⽚绕过
了国际标准的限制，在硬件层⾯留了后⻔，允许⽤户通过特定的协议指令⽆
限次修改 Block 0（即存放 UID 的地⽅）。
别⼈只要监听到了你的贴纸 UID，买⼀张⼏⽑钱的魔术卡，把你的 UID 写进
去，这张魔术卡在⻔锁看来，和你那枚正版贴纸没有任何物理区别。
红队展示缺陷：你们⽤这枚 F0387414L 贴纸当钥匙，让⻔锁 V1.0 只验证它
的 UID。然后⽤ Proxmark3 远程监听这个 UID，成功⽤第⼀包⾥的魔术卡克
隆并打开⻔锁。向教授证明：“只⽤ NFC 贴纸的 UID 当⻔锁就是找打。”
0.
蓝队展现技术：升级到⻔锁 V2.0。不使⽤ UID 验证，⽽是利⽤这枚贴纸内
部的存储空间（Data Pages），写⼊⼀段加密数据，并⽤ NTAG21X 芯⽚特有
的 32-bit PWD（密码保护）机制把这块空间锁死。刷卡时，读卡器必须先
发送正确的密码才能读取数据。因为魔术卡没有 NTAG 芯⽚的密码硬件状态
机，攻击者就算听到了密码，也⽆法在魔术卡上复现这个认证过程，从⽽实
现“防监听破解”！
1.
第⼆个包裹⾥还有⼀⼤包卡⽚，⼀个袋⼦⾥装着⼀堆蓝⾊的
⼩圆卡⽚，我记得之前很多中国的⼩区⻔禁都⽤这种卡⽚来
着，中间的圆形还能⽤暴⼒抠下来。然后有⼀包银⾏卡⼤⼩
的卡⽚，不过都是⽩⾊的卡⽚，其中有且只有⼀个正⾯印了
sparkfun ELECTRONICS RFID CARD


NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第30/65⻚
在实际的科研和项⽬开发中，完全没有必要把这⼏⼗张卡⽚⼀张张去查户⼝。 21
天的时间⾮常宝贵，我们要⽤最快的⼯程思维把它们分类归堆，各司其职。
你看到的这些形态各异的卡⽚（蓝⾊⼩钥匙扣、⽩⾊卡、带标签的贴纸），之所以
⻓相不同，只是因为它们的物理外壳（封装形式）不同，但脱掉外壳，⾥⾯的芯⽚
核⼼只有区区两三种。
以下是针对这⼀⼤堆杂物的秒懂分类指南以及你们如何借⼒打⼒利⽤它们：
1. 蓝⾊⼩圆卡⽚（⼩区⻔禁钥匙扣）
2. ⽩⾊银⾏卡（包括印着 SparkFun 的那张）
底层底细：这在业界叫 Abs Keyfob（钥匙扣标签）。你之前能⽤暴⼒把中
间的圆形抠下来，是因为它本质上就是把⼀圈铜线圈和⼀个沙⼦⼤的芯⽚，
⽤热熔胶封在⼀个⼩塑料圆⽚⾥，外⾯再套⼀个蓝⾊塑料壳。
⼤概率是什么：这种卡⽚在中国⼩区⻔禁中，80% 是⽼式的 ID 卡（低频
125kHz，如 EM4100 芯⽚），20% 是⽆加密的经典 IC 卡（⾼频
13.56MHz，如 Mifare Classic 1K）。
利⽤策略： * 让 EE 同学随便抓两个，⽤第⼀包⾥那个带线圈的低频天线
（ LF_ANT ）配合 Proxmark3 运⾏ lf search 。如果读出 ID 卡号，直接丢
⼀边不⽤管（因为你们的 NFC 课题是 13.56MHz ⾼频）。
如果 lf search 没反应，⽽⽤⾼频天线 hf search 读出来了，那它
就是普通的 M1 卡，可以充当你们⻔锁的备⽤钥匙。
底层底细：标准的 PVC ⽩卡。SparkFun 是国外著名的开源硬件商，它定制
的这张卡往往是极其标准的、⽤来给学⽣做实验的⽩卡。
⼤概率是什么：它们基本全都是 13.56MHz 的⾼频卡。不是 MIFARE Classic
1K（M1卡），就是 MIFARE Ultralight。
利⽤策略：它们是你们项⽬⾥最完美的“正版钥匙卡”备件。挑两张出来，
⽤你们的 ACR122T 读卡器读⼀下，记录下它们的芯⽚型号，留作⻔锁钥匙。

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第31/65⻚
3. 三个圆形 Tag（F0387414L、S2032661F 等）与⽅形⼤ Tag
极其⾼效的“⼀分钟盲筛法”（由 EE/IS 同学执⾏）
为了彻底结束分拣⼯作，你们只需要做⼀次抽样盲筛，把卡⽚分成两堆即可：
通过这个流⽔分拣，你们在 10 分钟内就能把这两⼤包不知名卡⽚，变成井井有条
的“项⽬实验资产”。此时，你们⼿⾥的武器库已经完全清点完毕，可以让你们的
Agent 正式开始疯狂输出 Python ⻔锁程序和攻击脚本的代码了！
底层底细：这些是 NFC 柔性抗⾦属标签/嵌体（Inlay）。那个⽅形⼤标签因
为⾯积⼤，天线线圈做得很⻓，所以在物理上的射频耦合距离最远（刷卡最
灵敏）。
利⽤策略：它们全部都是⾼频 NFC 标签（⼤概率是 NTAG21X 系列）。完全
不需要⼀⼀验证，因为它们对于⻔锁来说，功能和那枚 F0387414L 是⼀模
⼀样的。留着它们，等你们把⻔锁代码写好后，拿它们当不同⽤户的“钥匙”
来测试多⽤户管理即可。
第⼀步（分拣低频垃圾）：拔掉 HF_ANT ，插上 LF_ANT （带线圈的那
个）。在 Proxmark3 终端输⼊ lf search 。把那堆蓝⾊钥匙扣和⽩卡随便
拿⼏个放上去。
0.
如果能读出数据：这⼀类卡⽚直接贴上标签写“低频ID卡”，打⼊冷宫，
期末报告⾥只字不提。
第⼆步（挑选红蓝对抗的“⼦弹”）：
拔掉 LF_ANT ，插上 HF_ANT 。在终端输⼊ hf search 。把剩下的⽩卡和
圆形贴纸放上去。
1.
如果显示 MIFARE Classic ，就运⾏ hf mf cdetect （检测是不是魔
术卡）。如果是魔术卡，放进“红队改写卡（⼦弹箱）”；如果不是魔术
卡，放进“正版钥匙箱”。
如果显示 MIFARE Ultralight 或 NTAG ，放进“⼆代密码防御测试
箱”。

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第32/65⻚
最后⼀个包裹，是个HackRF的盒⼦，⾥⾯有个很酷的机器和
⼀根天线，以及⼀根microusb线、⼀个拓展坞。请问这个是
拿来⼲嘛的
这个包裹⾥装的，就是你们整个项⽬中技术含⾦量最⾼、最硬核的“降维打击”神
器——HackRF One（软件⽆线电，SDR）。
你看到的那个“很酷的机器”，⼤概率是安装了 PortaPack H2 扩展板的 HackRF
One（它带有⼀个彩⾊屏幕、⽅向键和⼀个旋钮，这让它脱离电脑也能独⽴⼯
作），那根天线则是⽤来接收和发射⽆线电波的。
在你们的 NFC 项⽬中，它的根本任务是：从纯物理的“信号与系统”层⾯，对
13.56MHz 的 NFC 磁场进⾏降维打击，实现⽆线电监听和主动射频⼲扰
（Jamming 防御）。
以下是它在你们 21 天项⽬⾥的核⼼⽤途和保姆级上⼿指南：
 1. HackRF One 是拿来⼲嘛的？
传统的读卡器（如 ACR122T）和攻击⼯具（如 Proxmark3）是“协议型”设备——它
们内部的硬件芯⽚已经写死了，只能识别符合国际标准（如 ISO14443）的“0”和“1”
数据包。如果空中出现⼀堆杂乱的波形，它们只会报错并直接丢弃。
⽽ HackRF One 是⼀个“数字眼睛和嘴巴”。它可以把天线周围任何频率（1MHz 到
6GHz，包含了你们的 13.56MHz NFC 频段）的⽆线电模拟信号，直接数字化变成
原始的 I/Q 采样数据（时域波形）输送给电脑；同时，它也能把电脑⾥计算出来
的数学波形，原封不动地通过天线发射出去。
 2. 在你们的项⽬⾥，它如何发挥威⼒？
这个设备是留给你们团队中 EE 同学（懂信号与系统）拿⾼分的绝对底牌：

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第33/65⻚
任务⼀：物理层波形截获与解调（EE 同学的主场）
任务⼆：实施“不可被监听破解”的物理防御（蓝队⼤招 - Active
Jammer）
这回应了你们最初的⽬标——如何让⻔锁系统“不可被监听破解”。普通的加密只能
防⽌别⼈复制，⽽ HackRF 能直接让别⼈的 Proxmark3 变瞎！
 3. 明天团队开⼯的第⼀步怎么做？
怎么做：把那根天线拧在 HackRF 上，使⽤ microUSB 线把 HackRF 连上电脑
（或者通过扩展坞）。在电脑上打开开源的⽆线电软件 GQRX 或 GNU
Radio。
看什么：将 HackRF 的接收频率对准 13.56MHz 。当你拿着钥匙卡去刷模拟
⻔锁（ACR122T）时，EE 同学可以在电脑屏幕的瀑布图（Waterfall）上赫然
看到⼀条瞬间亮起的能量谱线。
学术价值：你们可以把这段录制下来的波形导⼊ MATLAB。EE 同学可以通过
“信号与系统”的知识，画出完美的 ASK 100% 调制波形图。在期末 LaTeX 报
告中，放上这张由你们⾃⼰⽤ SDR 捕获并解调的物理波形图，教授的印象分
直接拉满。
防御逻辑：你们的⻔锁平时是静默的。当有合法⽤户靠近刷卡时，⻔锁开始
与卡⽚交互。
0.
主动检测与⼲扰：你们将 HackRF 部署在⻔锁旁边。利⽤ GNU Radio 脚本实
时监测 13.56MHz 的信道能量。
1.
定向爆破：如果⿊客带了 Proxmark3 偷偷贴在你们的⻔锁后⾯企图监听。
HackRF 会敏锐地捕获到磁场中多出了⼀个⾮法的耦合天线带来的阻抗扰动
（能量突变）。
2.
强⾏断连： ⼀旦触发防御阈值，HackRF 瞬间转为发射模式，全功率发射⼀
段 13.56MHz 的⽩噪声⼲扰信号。这股物理噪声会瞬间淹没⻔锁和卡⽚之间
的微弱信号，让⼀旁企图窃听的 Proxmark3 硬件直接过载并抛出 Missing
Sync （失去同步）错误，从物理层彻底掐断被监听的可能。
3.

NUS - Google Gemini 2026/7/14 21:51
https://gemini.google.com/app/a4e2e2eae35e413c 第34/65⻚
三个包裹已经全部盘点完毕，“军械库”已经完全展开。明天你们⼩组成员在寝室碰
头时，可以直接这样分⼯，开启 Phase 1：
有了这三包顶配硬件，加上你们软硬件贯通的背景，21天内做出⼀个让教授惊艳的
NFC 顶级项⽬完全在你们的掌控之中！
第⼀组（你 + CS/IS 同学）：把第⼆个包裹⾥的 ACR122T 读卡器插上电
脑。让你们的 Agent 给你们写⼀段 Python 的 pyscard 脚本，把⻔锁 V1.0
（只读 UID 就打印 Access Granted）的代码调通。
第⼆组（EE 同学）：
把第三个包裹⾥的 HackRF One 连上电脑，安装 SDR# (Windows) 或 GQRX
(Mac/Ubuntu)。让他们把天线拉⻓，尝试在软件⾥调到本地的 FM ⼴播频率
（⽐如新加坡的 93.3MHz 左右），如果能听到清晰的收⾳机⼈声，说明
HackRF 硬件完好、驱动正常。接着，让他们把频率调到 13.56MHz ，准备
捕获第⼀组同学刷卡时的波形。